ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/370143846

A machine learning-based resource-efficient task scheduler for heterogeneous computer systems

Article *in* The Journal of Supercomputing · April 2023

DOI: 10.1007/s11227-023-05266-4

CITATIONS
4

READS
182

4 authors, including:



Asad Hayat
Taiyuan University of Science and Technology
11 PUBLICATIONS **20** CITATIONS

SEE PROFILE



Muhammad Siraj Rathore
Capital University of Science and Technology
21 PUBLICATIONS **139** CITATIONS

SEE PROFILE

All content following this page was uploaded by Asad Hayat on 31 May 2024.

The user has requested enhancement of the downloaded file.

The Journal of Supercomputing
https://doi.org/10.1007/s11227-023-05266-4

A machine learning-based resource-efficient task scheduler for heterogeneous computer systems

Asad Hayat[4] · Yasir Noman Khalid[3] · Muhammad Siraj Rathore[1] · Muhammad Nadeem Nadir[2]

Accepted: 4 April 2023
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2023

Abstract

Heterogeneous computer systems are becoming mainstream due to their disparate processing and performance capabilities. These systems consist of different types of devices, i.e., central processing units (CPUs), accelerators, and graphics processing units (GPUs). In the heterogeneous computing environment, if one device is more powerful in terms of computing capability, the scheduling schemes generally favor the powerful device, and that device becomes overloaded, while the other device is underutilized. This load imbalance problem results in increased execution time. In this research, we propose load-balanced task scheduler combined with machine learning-based device predictor. The device predictor is used to predict execution time both on CPU and GPU devices, and a device with shorter predicted execution time is considered as a suitable device for that particular task. However, it may happen that a high fraction of tasks map only on one type of device since that device is considered as a suitable device for them. It is due to the fact that a task is mapped to one device (with lower predicted execution time), although it can be executed on the other device as well. In this context, one device may become overloaded, while the other device may be underutilized. To solve this problem of load imbalance, we use work-stealing-based task scheduler as part of our solution that allows an idle device to process tasks from the queue of another's device. In this way, we can avoid load imbalance, minimize the overall execution time of tasks, and maximize the device utilization and throughput. We evaluate the performance of our proposed solution into two stages. Firstly, we measure the error rate of our machine learning predictor using three different algorithms (i.e., random forest, gradient boosting, and multiple linear regression). We demonstrate that random forest performs better with marginal error rate. Secondly, we compare the performance of work-stealing task scheduler with other scheduling alternatives. Our results show that the proposed solution reduces execution time by 65.63%, increased resource utilization by 93.3%, and throughput by 65.5% in comparison with baseline scheduling schemes.

Extended author information available on the last page of the article

Published online: 20 April 2023

Keywords Heterogeneous computing · Machine learning · Gradient boosting · Work stealing · OpenCL

1 Introduction

Nowadays, heterogeneous computer systems are commonly used to execute computationally intensive computer programs. A heterogeneous computer system consists of multiple different processor types such as central processing unit (CPU) and graphics processing unit (GPU). Normally, a CPU device executes sequential programs with latency optimization, and GPU device executes parallel programs with throughput optimization [1–3].

Open compute language (OpenCL) is developed as an industry standard for data-parallel computer applications which can be executed on heterogeneous computer systems to achieve better performance [4]. We consider OpenCL framework in this research due to its portability and having wide range of supported computing devices.

In a heterogeneous system, it is required to decide that a job should be mapped on a CPU queue or a GPU queue. There are multiple scheduling techniques for the mapping of jobs to CPU and GPU device in the heterogeneous system [5]. The scheduler determines a specific data-parallel job that should be allocated to executing device, i.e., CPU or GPU [6]. However, it is not obvious that which type of processing device is suitable for a particular OpenCL application. In fact, some OpenCL applications may perform better on CPU, while others perform better on GPU.

Some researchers suggest mapping jobs on devices at run-time [7, 8]. The advantage of scheduling tasks on run-time is that the decision of job mapping is more efficient and can be adjusted at run-time, but it increases the complexity and higher the scheduling overhead. For instance, [9, 10] splits the code and maps each part on a suitable computing device.

A machine learning predictor can also be used to suggest a suitable device for a given job. In this regard, static and dynamic code features are widely used to characterize a given program [11, 12]. The machine learning model (i.e., predictor) is trained against static and dynamic code features to predict the suitable device (CPU or GPU) for a job. For instance, number of multiplications, number of float data types, number of functions, etc., are the static code features, whereas input workload is a dynamic code feature [13]. Most of the scheduling mechanisms schedule a single data-parallel program that depends on code splitting and requires application profiling [14].

The longer execution time of applications is the main concern while executing data-parallel applications. When only GPU executes jobs from the job pool, it takes a long execution time as the CPU is not taking part in application execution. The overall execution time of the job pool can be reduced by assigning jobs to both CPU and GPU simultaneously. However, generally CPU-only performs the management

of assignment of jobs to GPU and does not take part in the actual execution of programs, and this causes load imbalance between the computing devices [15].

Figure 1 shows the execution of programs on different processing devices. It shows that executing programs on CPU-only takes more execution time and under-utilization of processing devices as GPU remains idle. Executing all programs on GPU only also causes an increase in execution time and under-utilization of CPU device.

By assigning a program to a suitable device can reduce the execution time to some extent, but still the under-utilization of a processing device occurs. For instance, more jobs mapped on GPU which means CPU remains idle. Figure 1 shows that CPU becomes idle after 19.8 s, whereas GPU processing continues until 39.6 s. Although we have reduction in overall execution time as compared to other two cases, CPU remains idle for around 50% of time which is indeed inefficient use of CPU.

In this context, we believe that a scheduler which utilizes both CPU and GPU in a load-balanced manners can further reduce execution time. So, both CPU and GPU devices process load in parallel and complete the assigned tasks ideally at the same time. It can reduce the overall execution time of job pool, maximize throughput and device utilization.

In this research, we propose combining a machine learning-based predictor with a load balancer task scheduler. The predictor is used to predict the suitable device for a given job, whereas task scheduler ensures load balancing between CPU and GPU. For instance, if predictor suggests most of the jobs to execute on GPU, and GPU has a long queue, whereas CPU queue becomes empty, our load balancer task scheduler allows CPU to steal a job from GPU queue. The main contributions of our work are as follows:

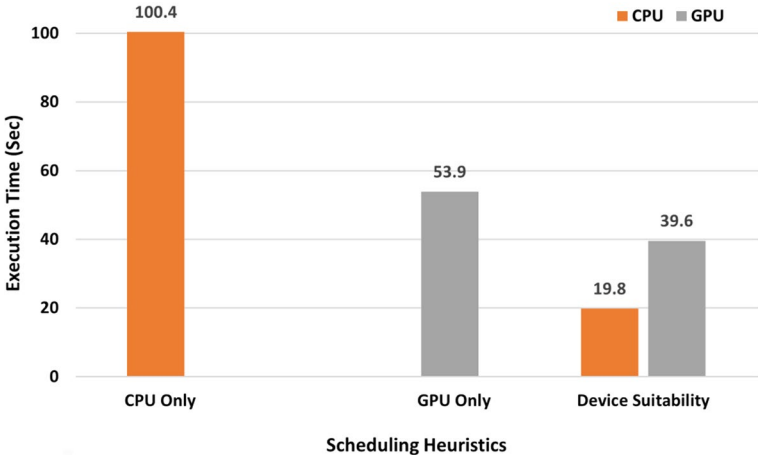


Fig. 1 Execution time of programs

1. A machine learning-based model development for the prediction of the program execution time on a CPU device and on GPU device. We compare the accuracy of three different machine learning algorithms (i.e., random forest, gradient boosting, and multiple linear regression) in this regard.
2. Combining the predictor with load-balanced task scheduler to reduce overall execution time of the programs in job pool and maximize the device utilization.
3. Analysis and performance comparison of different scheduling alternatives with our proposed solution.

The rest of paper is organized as follows: In Sect. 2, we present related work and make a comparison with our proposed solution. The design of proposed solution is present in Sect. 3, whereas Sect. 4 further elaborates components of the proposed solutions. In Sect. 5, we evaluate the performance of our solution and make a comparison with other scheduling alternatives. The paper is concluded in Sect. 6.

2 Related work

There are many studies that suggest different task schedulers for heterogeneous computer systems. However, there are some studies that propose machine learning predictor along task scheduler, whereas other studies focus only on efficient task scheduler. These studies are categorized into two subsections. Section 2.1 discusses those scheduling schemes where machine learning predictor is not the part of proposed solution, whereas Sect. 2.2 presents studies that use machine learning predictor as a part of proposed solution. Section 2.3 summarizes our findings from literature review and compares existing work with our proposed solution.

2.1 Scheduling schemes for heterogeneous computer systems

Heterogeneous dynamic self-scheduler (HDSS) [7] proposes a new method to divide workload among multiple processing devices. The proposed scheduler works into two phases, i.e., profiling phase and adaptive phase. In profiling phase, each processing device is assigned some loop operations in order to evaluate computation power of that device. In adaptive phase, remaining loop operations are assigned to these devices based on their performance in the profiling phase. In this way, the offered workload to a device is according to its computational capacity which is useful to minimize the overall programs execution time.

In another study [8], researchers propose a novel scheduler to map programs on CPU and GPU devices. The proposed scheduler estimates the execution of each application based on the applications execution time history and the remaining execution time of currently executing applications. Based on the estimated execution time, each application is scheduled to either CPU or GPU device.

The authors argue in [16] that it is difficult to manage communication and load-balancing issues in OpenCL programs. To solve the problem, a solution is proposed

which consists of static and dynamic load-balancing approaches. In static phase, kernel code is partitioned and mapped to different processing devices and execution time of each kernel code is noted. In dynamic phase, partitioned kernel queuing is adapted to achieve optimization.

Co-scheduling Asymptotic Profiling (CAP) [17] is two-phase scheduling method designed for heterogeneous computing. In first step, a small amount of workload is fairly assigned to both CPU and GPU devices and execution time is measured. In second step, partitioning of entire workload is performed, and the amount of workload is doubled on a faster device. (The faster device is determined based on the execution time from the first phase.) The process of increasing the workload continues for each subsequent partition. However, if the variance between the execution time of the current partition and the previous partition is less than a specified threshold, the workload is not increased for subsequent partitions.

In another study [18], authors extend OpenCL to a new programming framework, i.e., EngineCL. The framework is designed to simplify the programming of heterogeneous systems by offering scheduling and data management primitives for CPUs, GPUs, and FPGAs. The extension encompasses several features such as facilitating communication between the host and the FPGA, defining how kernel arguments are passed and enabling the use of different kernels for each device.

The work presented in [19] introduces multiple kernels on multiple devices (MKMD) that automatically maps multiple kernels on multiple computing devices. The system works in two phases. In the first phase, the system makes use of coarse-grained scheduling of kernels. In the second phase, the fine-grained workgroup-level partitioning is performed that utilizes the idle resources.

HSTREAM [20] is a directive-based language extension that supports programing stream of different applications in heterogeneous environment. It enables the programmers to assign parallel tasks to heterogeneous execution. For experimentation purpose, multiple programs of STREAM benchmark are executed. Authors claim HSTREAM as an attractive parallel programming framework that offers programming simplicity comparable to OpenMP. In comparison with our work, the scope of this study is different since they propose a new programming framework for parallel processing.

The authors propose an extension of Habanero-C programing system known as HC-K2H [21] that is a hybrid programing model implemented on Texas Instruments Hawking System-on-a-Chip (SoC) platform. The work-stealing algorithm is used for efficient utilization of processing devices (CPU and DSP cores) which is something similar to our work. However, in contrast, our solution combines machine learning model with work-stealing algorithm. Our aim is to predict a suitable processing device first for a task and then use work-stealing algorithm for further performance improvement.

The authors propose heterogeneous work-stealing (HWS) scheduler [22] based on work-stealing algorithm. The scheduler is executed on Heterogeneous Architecture Research Platform (HARPv2). HWS works on graph processing utilizing breadth-first search (BFS), single-source shortest path (SSSP), and PageRank (PR) graph processing kernels. The results of HWS are compared with static scheduling,

heterogeneous adaptive partitioning (HAP), and oracle self-scheduler. The results show that HWS improves up to 100% comparing with static scheduling, up to 17% comparing with HAP, and 1–3% comparing with oracle chunk. HWS uses work-stealing algorithm graph processing workload and schedules programs in heterogeneous environment, whereas our proposed solution evaluates execution time and combines machine learning with work stealing for optimization and device utilization.

2.2 Scheduling schemes based on prediction models

OpenCL is a parallel programming framework that can execute a program on multiple devices such as CPU, GPU, and FPGA. In this regard, *Smart Multi-Task Scheduling for OpenCL Programs on CPU/GPU Heterogeneous Platforms* [2] schedules multiple kernel programs on CPU/GPU devices at run-time in order to achieve best device utilization. Authors predict kernel speedup using SVM machine learning model. For this purpose, the static code features are extracted through clang and LLVM along with input data size for scheduling jobs. The authors report that their solution improves throughput by 1.25 times and turnaround time by 1.5 times as compared to existing alternative approach. In contrast, in our proposed solution, we compare three difference machine learning models to predict execution time on CPU and GPU devices. We explore that random forest is better with lower errors as compared to other alternatives of multiple linear regression and gradient boosting. We combine random forest predictor with work stealing job scheduling for better throughput and device utilization.

The researchers designed a classification framework [3] which is based on the device suitability. The framework predicts a computing device for an OpenCL program through a machine learning model. The machine learning model is trained with and without important features that are extracted from OpenCL programs. The important and influential features are selected through correlation analysis and tree-based feature selection technique. The machine learning model is evaluated, and F-measure of 0.91 and R^2 of 0.76 are reported.

The work presented in [11] estimates best application speedup for CPU/GPU devices using machine learning classifier based on dynamic code-feature extraction through LLVM framework. However, this approach is platform-dependent and does not work properly on complex programs. Furthermore, it takes more time as compared to static code features.

The researchers proposed a static feature and predictive modeling-based program [14] that partition OpenCL programs on CPU and GPU devices using SVM machine learning along with principal component analysis (PCA) for dimensionality reduction. The model is evaluated on GPU-friendly, CPU-friendly, and other benchmarks, while the proposed model predicts the execution time of jobs and schedules them to reduce the overall execution time of the job pool.

Troodon [13] is a machine learning-based scheduling model, based on E-OSched which allocates the low computation requirement OpenCL programs to CPU and high computation programs to GPU in a heterogeneous architecture. The machine

learning model predicts the execution speed of a task and E-OSched maps jobs to either CPU or GPU device. E-OSched ensures load balance by inspecting computational requirement of a job and computing potential of a device. In this way, Troodon considers job requirement, device suitability and speedup factors while maintaining a load balance on computing devices. Troodon is evaluated for data-parallel applications, and it is demonstrated that execution time is reduced by 2.85 times as compared to CPU-only, whereas throughput of 40% is increased as compared to CPU-only. Troodon considers machine learning for predicting speedup of processing devices and uses E-OSched for scheduling purposes, while our proposed approach predicts the execution time using random forest and uses work-stealing algorithm for scheduling purposes.

The researchers [23] present a run-time model that schedules the tasks on a suitable CPU or GPU device. The model predicts at run-time whether to merge or schedule separately OpenCL programs in a job pool using a decision tree machine learning model. The model is based on both static code feature and dynamic code features. The model first separates the kernel to CPU device and GPU device based on estimated device affinity and then determines whether to merge the kernel or not. The merging of programs is done through the JIT compiler and scheduling by a thin run-time layer.

Resource Aware Load Balancer for Heterogeneous Cluster RALB-HC [5] is a supervised machine learning-based approach that distributes workload in multi-node heterogeneous computing environments by considering computing capabilities of nodes along the computing needs of applications. Furthermore, this model investigates device suitability, expected speedup, and load balancing for the mapping of jobs in a heterogeneous environment.

Authors argue in [24] that machine learning classifiers do not perform well in high-density computing requirements because of execution of instructions in a sequential fashion. To fix the problem, they propose a machine learning model that exploits the hardware parallelism. In this context, GPUs are used for training purposes, and FPGAs are used as inference models (using a model converter). The approach is evaluated through neural networks models.

The researchers propose an approach [25] to optimize the performance of applications along energy efficiency on a heterogeneous platform. AI heuristics are employed for parameter exploration along a decision tree regression model to evaluate performance and energy consumption. The approach aims to determine the best workload distribution on processing devices, the optimal number of processors required, and the most effective scheduling strategy. For experimentation purpose, GPU and Intel Xeon Phi processing devices are used. In their findings, they claim that machine learning-based solution can predict performance per joule 100 times faster than normal execution. This approach is using machine learning for predicting performance of the system and neither use any scheduling algorithm nor predict execution time on processing device. In contrast, our proposed solution uses work-stealing strategy for load balancing and machine learning for predicting execution time.

Heterogeneous architecture has different processing units (CPU and GPU) that cause load imbalance due difference in their processing power. HBalancer [26] schedules programs on CPU and GPU while calculating load difference through execution time. HBalancer compares their results with device suitability and round robin algorithm considering load balancing. The results shows that HBalancer outperforms device suitability by 2.59 times and round robin by 4.64 times in term of execution of programs. HBalancer predicts execution time through machine learning models (extra trees regressor and light gradient boosting) and designed their own scheduler to reduce imbalance between processing devices. In contrast, our proposed approach evaluates the execution time which is predicted through random forest model and uses the work stealing for scheduling programs.

2.3 Analysis of existing literature

After comprehensive analysis of different approaches, there are multiple techniques for scheduling in heterogeneous environment. Some scheduling techniques creates load unbalancing, while scheduling as CPU-only manages kernel execution and does not take part in actual computation [17, 27, 28]. The CPU waits until GPU completes their execution that is not desirable. In other studies, [19, 29] split program code between CPU and GPU device. Similarly, [11] does not consider the load balancing for mapping jobs to the scheduler.

In [15, 30], authors map a single program to either CPU or GPU device, while creates a job scheduler to schedule jobs from job pool in heterogeneous system considering reduction in overall execution time of jobs, increase in throughput, and utilization of processing device. Predictive machine learning modeling is also an effective technique for optimizing parallel programs, and several learning models are trained [12, 23, 31]. However, we believe following are the limitations of the existing work:

1. Most of the scheduling techniques require code splitting. They split the program to execute on CPU and GPU. This code splitting results in additional time overhead.
2. Some researchers map a single job in a heterogeneous environment that causes time overhead.
3. To the best of our knowledge, no prior work evaluates the combination of machine learning model and work-stealing load balancer in a heterogeneous computer system.

3 Design Of Proposed Predict Device Load Balance (PDLB) Task Scheduler

In a heterogeneous computing environment, a task scheduler can map a task either on a CPU or GPU device. Ideally, a device with a shorter execution time for a particular task should be selected (i.e., suitable device). However, it is difficult for a

scheduler to make a device suitability decision because of two reasons. Firstly, the estimated execution time for a particular device is not known in advance. Secondly, there are variety of tasks from a pool of jobs [4] having many different types of programming attributes (e.g., total number of control statements, total number of functions, etc.) which may or may not favor a particular device (i.e., CPU or GPU).

Generally, programmers use the default scheduling strategy for mapping tasks on a heterogeneous system [3]. In this strategy, the CPU executes the serial portion (host part) of the program, while GPU runs the parallel portion (kernel part) of the program. This strategy causes CPU idle as all computation is performed by GPU. In such a situation, a GPU may become overload and results in high execution time for many tasks. In addition, the CPU remains idle (consuming power for nothing) during this time which is inefficient use of resource. Therefore, we believe it is challenging for the developers to map tasks on a heterogeneous system in an optimal way.

In this work, we aim to design a task scheduler that could predict a device with a shorter execution time. However, it might be possible that a high percentage of tasks map only on a one device which is predicted as a suitable with shorter execution time. It may result in an overloaded device, while the other device might be under loaded at that time. To avoid such a situation and make an efficient use of both CPU and GPU, we suggest a load-balancing strategy which works in the combination of our machine learning predictive model. The details of both predictive model and load balancer are given below.

3.1 Device selection using machine learning-based predictive model

The architecture of the proposed solution is shown in Fig. 2. The user submits data-parallel OpenCL programs and then proposed scheduling system assesses the submitted jobs. Our machine learning-based predictor predicts the execution time of a task both on CPU and GPU device. The device with shorter predicted time

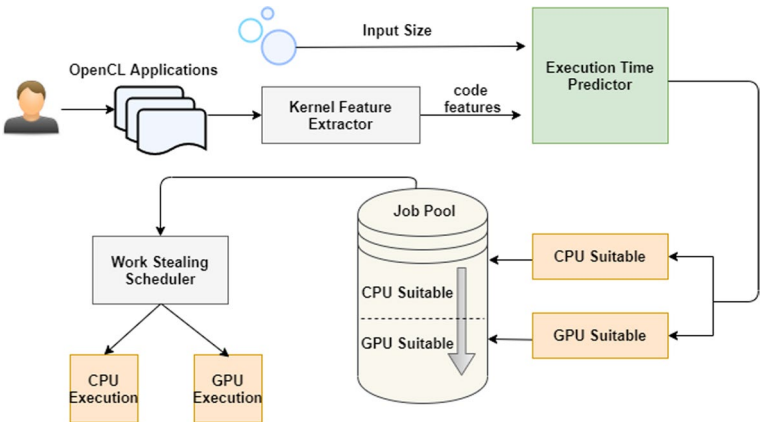


Fig. 2 Architecture of predict device load balance (PDLB) proposed solution

is selected for execution of a task. It is shown as a CPU suitable or GPU suitable in Fig. 2. To predict the execution time, the predictor is provided with two types of inputs. The first is the input data size of an application. The second is the code features (program attributes). We use the static code analyzer to extract the code features and then find out correlation between the features through correlation analysis. The purpose is to select only important and influential features for the training of our predictor. In short, the static code features and the dynamic workload (i.e., input size) of the jobs are used for predicting the execution time of each job on both CPU and GPU. The details of predictor training and testing are presented in Sect. 4.

After selection of a suitable device, the task is placed on an associated queue for that device. We have one queue for CPU suitable tasks, whereas the other queue is for GPU suitable tasks. It is now responsibility of work-stealing scheduler [6, 32] to select a task from these two queues for execution either on CPU or GPU. We use work-stealing scheduler since it is useful to balance the offered load between CPU and GPU. The load balancing leads to reduce the overall execution time of the submitted jobs and efficient utilization of the processing devices (i.e., CPU or GPU). The details of work-stealing scheduler are presented in Sect. 3.2.

It is important to mention that although the concept of combining a predictor and a scheduling algorithm is not new, however, according to the best of our knowledge, no one combined a predictor with work-stealing scheduler as suggested in Fig. 2

3.2 Load balancer

Work stealing is a scheduling algorithm that is widely used for the mapping of programs in heterogeneous environment having multi-core processing devices [6, 32]. The basic theme of the work-stealing scheduler is distributing the workload on the idle processing devices which ultimately causes the reduction in overall execution time of jobs and efficient utilization of the processing devices.

In work-stealing scheduler (Fig. 3), each processing device have a double-ended queue (i.e., deque) to hold the tasks assigned to it. The jobs in the deque are sorted in the descending order on base of their execution time. A processing device takes one job at a time from the associated deque. When a processing device executes all their assigned jobs and the deque of processing unit becomes empty, the device becomes thief and steals a job from the bottom of another processing device deque (which is known as victim processor). When the thief processor executes the job and more jobs are present in victim processor deque, the thief processor steal another job from the bottom of deque (last in first out). This process remains until all the available processing devices executes all the available jobs. It is important to mention that stealing of jobs starts only when a processing device has executed all their assigned jobs and then becomes idle. In our work, the concept of work-stealing scheduling algorithm is used for load balancing between CPU and GPU devices. Each device has an associated deque. CPU deque is created for the jobs to be executed on CPU and GPU deque is created for the jobs to be executed on GPU. However, we believe that in most of the cases the CPU is not as much loaded as a GPU

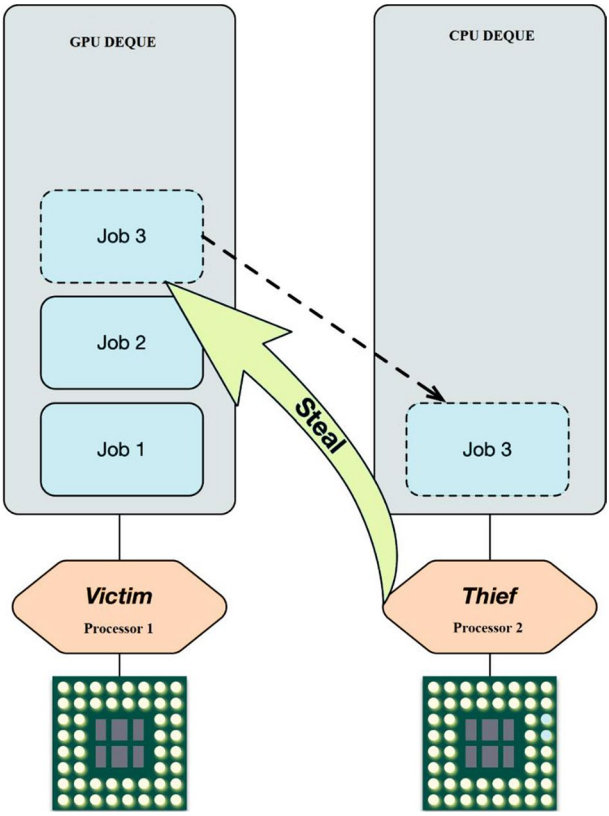


Fig. 3 Work-stealing scheduler [33]

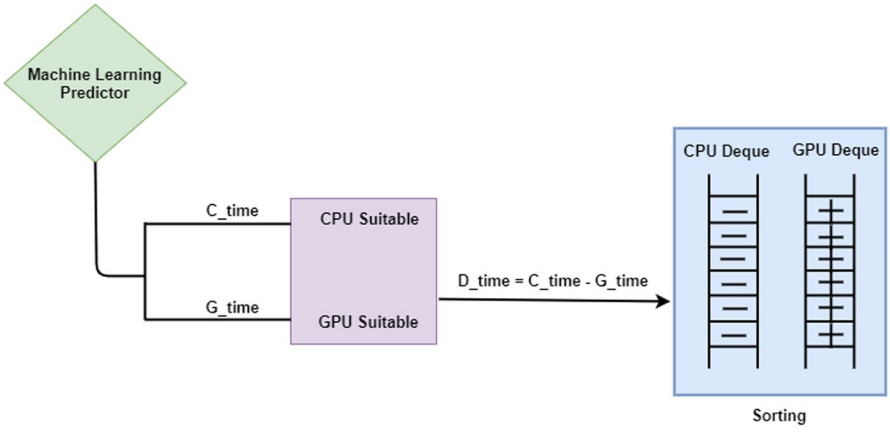


Fig. 4 Work-stealing scheduler for load balancing between CPU and GPU devices

[13] and work-stealing algorithm allows to pick up a job from GPU deque (at the end) for execution on CPU.

The proposed system is shown in Fig. 4. First, the execution time of jobs on CPU as well as on GPU is predicted through machine learning model. For finding the suitable device for a job, the predicted execution time of GPU (G_time) is subtracted from predicted execution time of CPU (C_time). The difference of C_time and G_time is known as D_time.

$$\text{D_time} = \text{C_time} - \text{G_time} \tag{1}$$

The D_time consists of both positive and negative values. A job having negative D_time value is considered as CPU suitable as it has less predicted execution time on CPU. A job having a positive D_time value is GPU suitable as it has more execution time on CPU. A job having D_time value equal to zero is considered as CPU suitable to break the tie.

The jobs sorted on the bases of D_time are moved to the job pool where two deque are maintained. The jobs with negative D_time values are placed on CPU deque, whereas jobs with positive values are placed on GPU deque in descending order. When both devices have jobs in their deques, a job in front of a respective deque is selected for execution. Thus, both CPU and GPU work in parallel by picking up jobs in order from their associated deques. However, when a processing device completes its execution of all assigned jobs, it steals the jobs from another processing device. For instance, suppose CPU completes the execution of the jobs available in the CPU deque, and GPU has jobs remaining in the GPU deque, the CPU will become thief and steal a job from the end of the GPU deque. Although CPU is not selected, a suitable device (with higher predicted execution time than GPU execution time) for the steal job since the job is picked up from GPU deque; however, GPU is busy at the moment in processing of other jobs, and it is likely that last job would be completed in parallel on CPU. In this way, we aim for the reduction in overall jobs execution time through efficient resource utilization.

4 Machine learning-based prediction model

This section covers the details of the machine learning prediction model. The first step is the selection of OpenCL applications and then predict whether CPU or GPU is a suitable device for that particular application. We select applications from two benchmark suites, i.e., Polybench benchmark and AMD benchmark suite. Polybench and AMD are set of computation kernels used to test the performance of compilers and related applications, such as matrix multiplication, 2D convolution, 3D convolution, or linear equation solver. Applications of Polybench are considered as a core of many applications for high-performance like image processing [34]. The details of selected applications and prediction model are presented in following subsections.

4.1 Application dataset generation

We have selected 20 applications in total from both Polybench and AMD benchmarks [19, 35]. These benchmarks are commonly used by many researchers for experimentation and analysis purposes [3, 36–38]. In addition, our aim is to include variety of workloads, and these 20 applications belong to different domains such as linear algebra and image processing. These applications are executed with different input data sizes both on CPU and GPU. The details of benchmark suit, their application, and input size are given in Table 1. A total 13 applications from Polybench and seven applications from AMD suite are executed. There are 1165 jobs in total which are executed on both CPU device and GPU device. The CPU time is the execution time of the application on CPU device, whereas the GPU time is the execution time of application on GPU device.

The results of our application testing on CPU and GPU are as follows. We discover that FDTD-2D application have less execution time on GPU device at every input size. The ATAX application have less execution time on CPU device at every input size (starting from 1 and maximum of 22,000 as shown in Table 1). For all other applications, less execution time on CPU at some input sizes is observed, whereas less execution time on GPU device is observed for some other input sizes. We generate two datasets with input size and execution time of the program. One dataset contains input sizes and corresponding execution times on

Table 1 Detail of benchmark suite and input size

Serial #	Suits	Benchmark applications	Input data size
1	Polybench	GEMM	1–4600
2	Polybench	3MM	1–4600
3	Polybench	GESUMMV	1–14,500
4	Polybench	ATAX	1–22,000
5	Polybench	2MM	1–22,000
6	Polybench	MVT	1–15,500
7	Polybench	BICG	1–22,000
8	Polybench	CORRELATION	1–2800
9	Polybench	COVARIANCE	1–2600
10	Polybench	FDTD-2D	1–12,000
11	Polybench	GRAMSCHMIDT	1–8800
12	Polybench	SYR2K	1–2200
13	Polybench	SYRK	1–3000
14	AMD	Binomial option	1–20,000,000
15	AMD	Bitonic sort	512–20,000,000
16	AMD	Discrete cosine transformation	1–8192
17	AMD	Fast walsh transform	1–100,000,000
18	AMD	Floyd warshall	1–15,500
19	AMD	Matrix multiplication	128–12,500
20	AMD	Matrix transpose	1024–10,000,000,000

CPU device, whereas the other dataset contains the program execution times on GPU device for the same list of input sizes.

4.2 Feature extraction

In addition to input size and execution time, the program features are also part of our dataset. The static code analyzer is designed for the extraction of features from the kernel code of OpenCL programs of two benchmark suite, i.e., Polybench and AMD. The extracted features set shows the behavior of the programs. The aim of the extraction of features from the code is to collect the attributes of the OpenCL programs from both benchmarks. The Clang (front end) compiles the code of OpenCL programs to ensure the code is error free. The LLVM passes extracts the code features. Figure 5 shows the complete methodology of extraction of the features from the OpenCL programs.

In this work, we have extracted 22 features from the OpenCL application programs. These features consist of number of loops, number of int data types, number of float data types, total number of functions, etc. Table 2 demonstrates all the extracted features from OpenCL programs through clang and LLVM.

Two datasets are generated from the static code features and input size along with the execution time of the program on processing devices. One dataset contains the code feature set, input size and the program execution time on CPU device, and the other datasets contain the code feature set, input size and program execution time on GPU device. These datasets (after feature selection as explained below) are used for training and testing of machine learning predictor. The details regarding training and testing of different machine learning models are presented in Sect. 5.

4.3 Feature selection

Our next task is to identify more relevant features out of these 22 extracted features. The feature selection means to find out importance of features by finding correlation of each feature with another feature. The selection of important feature is very crucial because it causes reduction in over-fitting improves the accuracy and reduces the training time of the classifier. We use Pearson correlation [32] which is a filter-based correlation analysis method and widely used for finding correlation between features.

According to Pearson correlation method, the two features are correlated if change in the value of one feature causes change in another feature's value. If increase in one feature's value causes increase in other feature's or decrease in one feature's value causes a decrease in another feature's value, the two features are correlated positively. Similarly, if decrease in the value of a feature causes increase in other feature's value or increase in one feature's value causes decrease in another feature's value, the two features are



Fig. 5 Code feature extraction

Table 2 Static code extracted features

Serial number	Feature name
1	Total number of return statement
2	Total number of control statement
3	Total number of allocation instruction
4	Total number of load instruction
5	Total number of store instruction
6	Total number of multiplication (Float data type) operation
7	Total number of multiplication (Integer data type) Instruction
8	Total number of division (Float data type) instruction
9	Total number of division (Integer data type) instruction
10	Total number of condition check instruction
11	Total number of addition (Float data type) instruction
12	Total number of addition (Integer data type) instruction
13	Total number of subtraction (Float data type) instruction
14	Total number of subtraction (Integer data type) instruction
15	Total number of function call instruction
16	Total number of functions
17	Total number of blocks
18	Total number of instructions
19	Total number of float operation
20	Total number of integer operation
21	Total number of loop operation
22	Total number of loops

correlated negatively. Using positively correlated features for training purposes causes lower prediction accuracy as demonstrated in [33]. Therefore, in our study, we select negatively correlated features.

In this study, we use Heatmap [34] which is a tool to visualize the correlation between features. Figure 6 shows the matrix of the correlation of features which are listed in Table 2. The two features having value 1 or tends to 1 are positively correlated, and features having values negative or tends to a negative value are negatively correlated. Out of 22 features, we find that eight features have positive, while 14 features have negative correlation with other features.

The features having correlation value less than or equal to -0.1 are selected and used by the machine learning model for predicting execution time on CPU and GPU devices. The selected features are shown in Table 3.

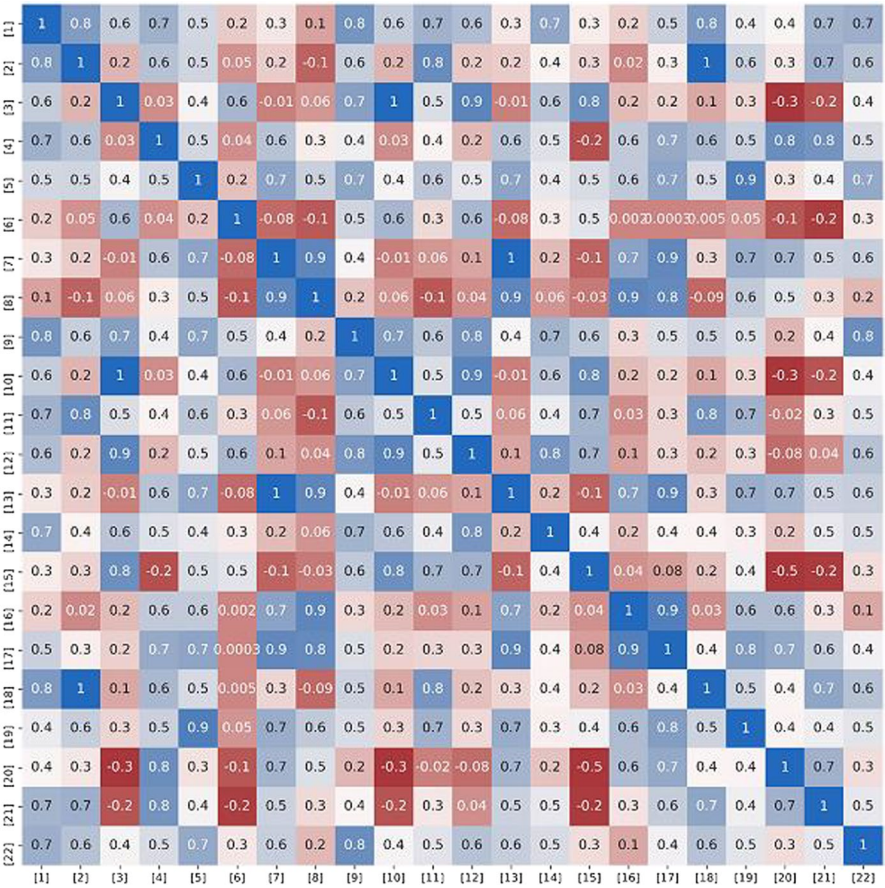


Fig. 6 Correlation analysis of OpenCL features

Table 3 List of selected features

Serial number	Feature name
1	Total number of allocation instruction
2	Total number of multiplication (Float data type) operation
3	Total number of division (Float data type) instruction
4	Total number of condition check instruction
5	Total number of subtraction (Float data type) instruction
6	Total number of function call instruction
7	Total number of instructions
8	Total number of integer operation
9	Total number of loop operation

4.4 Machine learning model selection

The next step is to use our datasets (with selected code features and input size) for the training of machine learning model. We use supervised machine learning models. The supervised machine learning has further two types: classification and regression. Classification is type of supervised learning in which the output class label is categorical (discrete), while the regression has numerical (continuous) output class label. In this research, regression models are used since our output class labels are CPU execution time, and GPU execution time are numeric values.

After the preprocessing completes, feature selection is applied on the dataset, and important and influential features are selected from the available extracted features for training and testing of the machine learning models. We use three different machine learning models in this regard, i.e., multiple linear regression, random forest regression, and gradient boosting regression models.

We execute applications both on CPU and GPU devices and record their execution times. This information is used to create two datasets, i.e., CPU dataset and GPU dataset. The CPU dataset contains values of nine important features, input size, and execution time on CPU device, whereas GPU dataset contains same nine features, input size, and execution time on GPU device. These datasets are used for training and testing the machine learning model for the prediction of the execution time of programs on CPU device and GPU device. The shape of datasets is 1165 tuples (rows) and 11 columns (nine selected features from code as listed in Table 2, input size and execution time). We train two models, i.e., CPU model and GPU model. The CPU model is trained on CPU dataset to predict the execution time on CPU, whereas GPU model is trained on GPU dataset to predict execution time of programs on GPU. We split our dataset (either CPU or GPU model) into two parts with the ratio of 20% and 80%. Table 4 shows that 80% of the dataset is provided to machine learning model for training purpose. The remaining 20% is used for testing and evaluating the trained model.

5 Results and discussions

We first discuss our experimental setup in Sect. 5.1, and then the results are presented in Sect. 5.2 and 5.3, respectively. In Sect. 5.2, we measure the error margins of our machine learning predictor. For instance, the difference between the predicted execution time on a device (CPU or GPU) and actual execution time that depicts the

Table 4 Detail of dataset used for prediction

Set	Instances	Percentage
Training set	931	80
Testing set	234	20
Complete dataset	1165	100

error. In Sect. 5.3, we compare the performance of work-stealing scheduling algorithm (as a part of proposed solution) with other scheduling schemes.

5.1 Experimental setup

For the implementation of the machine learning model, the Python programming language is used. We use PyCharm version 2019.2.3 professional edition toolkit and Jupiter notebook IDE along with Python version 3.6. As mentioned earlier, we have evaluated different regression models which are multiple linear regression, random forest, and gradient boosting.

The hardware specifications of the computer system used to implement task scheduler are provided in Table 5. It is a CPU (Intel)-GPU (NVIDIA) system with Linux Ubuntu 16.04 Operating system installed.

5.2 Machine learning predictor

The concept of performance evaluation in regression is that comparing the predicted values with the actual values. The predicted execution time on CPU device using all three machine learning models is compared to the actual execution time as measured manually and added as actual execution time in dataset.

Figure 7a–f shows the correlation of actual and predicted values of machine learning models. When the actual and predicted values are overlapped, it indicates that the model has predicted the execution time with maximum accuracy. In Fig. 7a–c, we compare actual and predicted values (of execution time) on CPU device using random forest, gradient boosting and multiple linear regression models, respectively.

Table 5 Hardware specification of computer system used to implement task scheduler

Device	CPU	GPU
Architecture	Haswell	Kepler
Processor number	Intel Core i5-4460	GeForce GTX 760
Number of cores	4	1152
Number of thread	4	–
Memory	8.00 GB	2 GB
Memory bandwidth	25.6 GB/s	192.2 GB/s
Performance	409.6 GFLOPS	2257.9 GFLOPS
ISP	32	2
Memory speed	1600 MHz	6.0 Gbps
OpenCL SDK	Intel SDK for OpenCL 2016	CUDA 8.0
Compiler	GCC 5.4.0	Nvcc

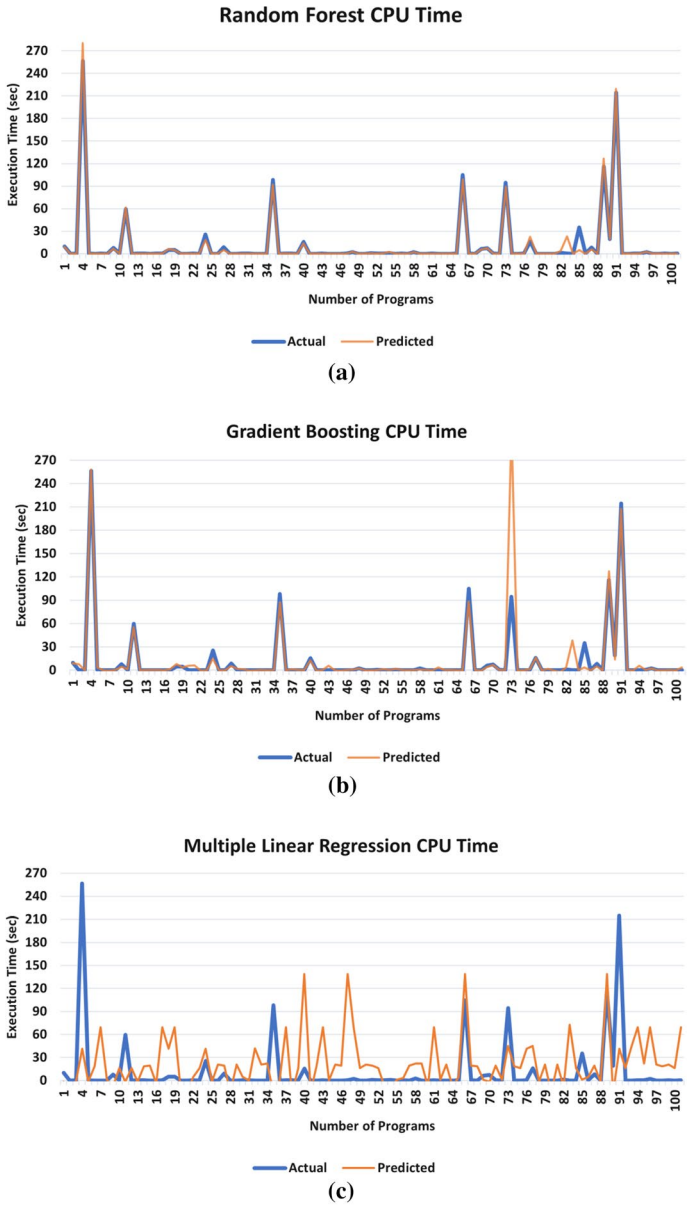
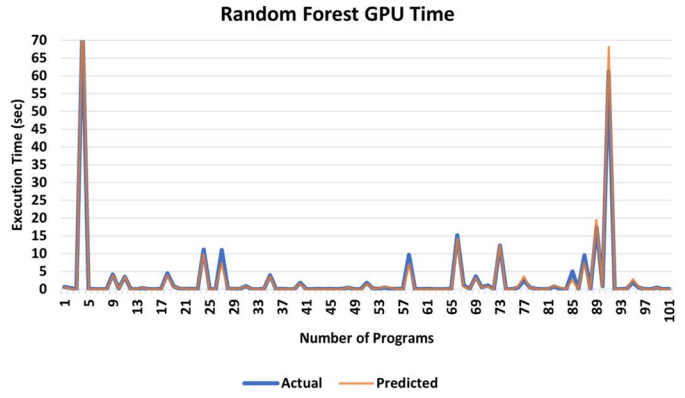
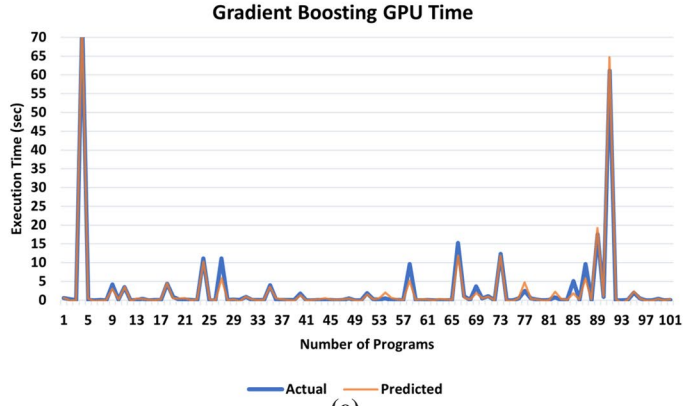


Fig. 7 **a** Random forest CPU time, **b** Gradient boosting CPU time, **c** Multiple linear regression CPU time, **d** Random forest GPU time, **e** Gradient boosting GPU time, **f** multiple linear regression GPU time

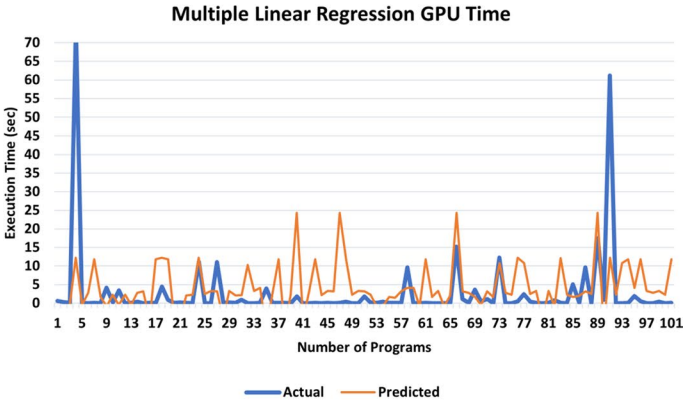
Similarly, in Fig. 7d–f, we compare actual and predicted execution time on GPU device instead. In both cases, it can be noticed that actual and predicted curves are overlapped mostly in case of random forest and gradient boosting, whereas a



(d)



(e)



(f)

Fig. 7 (continued)

clear difference between predict and actual values can be noticed for multiple linear regression. These results indicate that prediction of random forest and gradient boosting is better than multiple linear regression. However, there is no clear difference noticed between the accuracy of random forest and gradient boosting.

To get better insight about the accuracy of the machine learning models, we measure the error rate of the predictive machine learning models for regression by evaluating multiple performance metrics like R^2, root-mean-square error (RMSE), and mean absolute error (MAE).

Mean absolute error (MAE) shows the difference of the actual values and predicted values. It is the average absolute difference on the dataset. The formula of mean absolute error is:

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N} |yi - \hat{y}| \tag{2}$$

N—total number of tuples, Yi— Actual value, ŷ— Predicted value of y.

The MAE results are shown in Table 6. In the case of mean absolute error, the lower the value, the precise the model is and vice versa. In our prediction analysis, the random forest has the smallest values as compared to both gradient boosting and multiple linear regression.

Root-mean-square error is the square root of the mean square error.

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\sum_{i=1}^{N} (yi - \hat{y})^2} \tag{3}$$

It is the standard deviation of the prediction error from the line of best fit. In case of root-mean-square error, the smaller the value, the lower the error in the model and vice versa. Table 6 shows the comparison of the values of the RMSE of the machine learning models. It can be noticed that random forest performed the best with minimum value of 1.63 for GPU and 8.68 for CPU.

R^2 is coefficient of determination. It shows the coefficient that how accurate the predicted values fit as compared to the actual values. R-squared is normally in the range of 0 and 1. The higher the R-squared value, the better the model is, and smaller the R-squared value, inferior the model is. The formula of the R-squared is

Table 6 Machine learning models errors for CPU and GPU devices

Error type	Machine learning models					
	Multiple linear regression		Random forest		Gradient boosting	
	CPU	GPU	CPU	GPU	CPU	GPU
Mean absolute error	38.2	7.56	2.103	0.353	3.94	0.676
Root-mean-square error	85	16	8.68	1.63	17.66	2.018
R-squared	0.157	0.166	0.99	0.99	0.96	0.98

$$\mathrm{R-Squared} = 1 - \frac{\sum (yi - \hat{y})^2}{\sum \left(yi - \bar{y}\right)^2} \tag{4}$$

$\bar{y}$—*Mean value of y.*

R^2 is in the range of 0 and 1. The 0 mean 0% and 1 mean 100% but the percentage does not show the accuracy. It only indicates the fitting of actual and predicted values. Table 6 shows the comparison of R^2 values of multiple linear regression, gradient boosting, and random forest models. In our case, the random forest and gradient boosting models fits very accurately as compared to multiple linear regression. However, the random forest is slightly better than gradient boosting models. But multiple linear regression model is almost 88% lower than both models. The poor performance of multiple linear regression is likely because the data are not in linear form.

Our results demonstrate that random forest provides more accurate predicted values as compared to gradient boosting and multiple linear regression. Therefore, predicted execution time by random forest model is further utilized for the designing of task scheduler. To be more precise, in our proposed solution, we combine random forest machine learning model (for device prediction) with work-stealing task scheduler.

5.3 Performance evaluation of work-stealing-based task scheduler

As shown in Sect. 3.2, after predictor's decision, tasks are placed either on CPU queue or GPU queue. It is then responsibility of task scheduler to select a program for processing on a device. In this context, we compare the performance of work-stealing algorithm with other scheduling alternatives. It is important to mention that work-stealing algorithm itself is not our contribution since the algorithm is the part of the existing literature. However, we propose work-stealing algorithm as a part of our proposed solution. We believe that work-stealing algorithm is well-suited in our scenario where CPU becomes idle (more likely) after running the host part of a program, and work-stealing makes it busy by selecting jobs from GPU queue. We describe some other scheduling alternatives as well and make a performance comparison of these alternatives with work-stealing-based task scheduler.

5.3.1 CPU-only

In CPU-only, all programs are executed only on the CPU device without any evaluation that a program is CPU suitable or GPU suitable. (GPU remains idle.) It is considered as a baseline and is used as standard for the evaluation of the proposed work-stealing scheduler [13].

5.3.2 GPU-only

In GPU-only, all the programs are assigned to only GPU device, and in this case, CPU remains idle [13]. The GPU-only is also considered as baseline and standard for evaluation of the work-stealing scheduler.

5.3.3 First come first serve

In first come first serve (FCFS) scheduling, programs are processed in the order of arrival, and a free device is used for the processing of the program. For instance, both CPU and GPU are free in the beginning, and program P1 is assigned to CPU. Now, the only free device is GPU, and therefore, program P2 will be processed on GPU and so on.

5.3.4 Device suitability

In device suitability evaluation, the programs are assigned to their suitable devices. Suitable device means a job has less execution time on that device [13]. It is different than our proposed solution since we combine device suitability with work-stealing algorithm. In device suitability only, it is not possible to select a job from another queue. For instance, it is not allowed to select a job from GPU queue for execution on CPU device (although the CPU is idle at that time).

5.3.5 Alternate assignment

In alternate assignment, the programs (in a queue) are first shuffled and then assigned alternatively to the CPU and GPU processing device [13]. For instance, first program will be assigned to CPU, second will be assigned to GPU, third to CPU, fourth to GPU and so on. In this research, the programs are randomly shuffled many times. We have evaluated many different combinations in this regard. When we shuffle programs once, it is called as Alternate Assignment 1 (AA1), and when shuffled twice, it is called Alternate Assignment 2 (AA2). Similarly, we have tried Alternate Assignment 3 (AA3), Alternate Assignment 4 (AA4), and Alternate Assignment 5 (i.e., AA5 shuffled five times before assignment on a device).

5.3.6 Input size

All the jobs are sorted in descending order based on their input sizes. Half of the jobs from the top of the queue are assigned to GPU device, while the remaining jobs are assigned to CPU device. The job having largest value of input size is executed on GPU device, while the job having smallest input size is executed on CPU device [2].

The performance of work-stealing-based task scheduler is compared against aforementioned scheduling alternatives. In this research, total 120 programs are selected from two benchmarks suits polybench and AMD. The job pool contains 35 CPU suitable jobs and 85 GPU suitable jobs that are determined by predictor. CPU

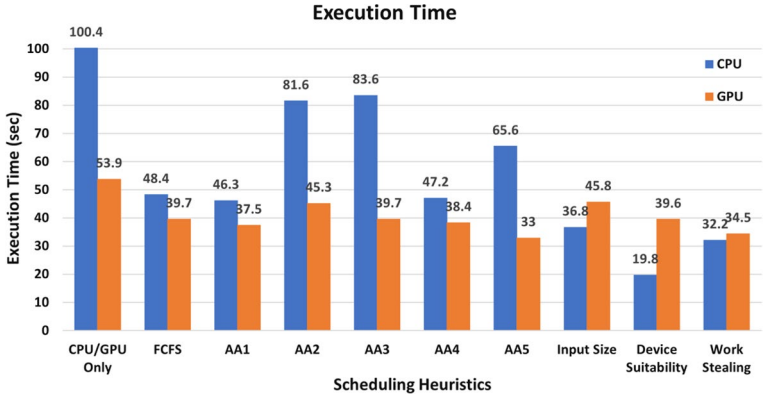


Fig. 8 Execution time of scheduling heuristics

suitable means predicted execution time of job is lower on CPU (as compared to predicted execution time on GPU), whereas GPU suitable means predicted execution time of job is lower on GPU. Three performance metrics are used for evaluation which are execution time, throughput, and utilization of devices. Figure 8 provides the comparison of all aforementioned scheduling heuristics in terms of execution time. This is the total execution time taken by either CPU or GPU to execute all assigned program according to a specific scheduling heuristic.

Figure 8 shows that all assigned tasks are completed in 34.5 s in case of work-stealing-based scheduler which depicts the overall best performance as compared to other approaches. The CPU bar for work-stealing-based scheduler shows that all tasks assigned to CPU are completed in 32.2 s, whereas all tasks assigned to GPU are completed in 34.5 s. It means that after 32.2 s CPU becomes idle, but GPU continues job execution that completes at 34.5 s. Thus, all assigned tasks are completed in 34.5 s which is the best result in comparison with other approaches. The difference between CPU and GPU execution time is marginal for work stealing, whereas it is quite significant for other approaches. It is because of load-balancing attribute of work-stealing algorithm.

We also notice that CPU execution time of work stealing is lower than all other approaches except device suitability where execution time is 19.8 s as compared to 32.2 s for work stealing. However, the load is not balanced on device suitability, and it results in a higher execution time for GPU device. It is 39.6 s for device suitability as compared to 34.5 s for work-stealing-based scheduler. It means work stealing is better in overall performance since it completes all tasks in 34.5 s, whereas device suitability takes 39.6 s. Similarly, GPU execution time of proposed solution is the lowest except the alternate assignment 5. The alternate assignment 5 has lower execution time on GPU device but more execution time on CPU device as compared to the proposed scheduler. As a result, the overall performance of proposed solution is better.

Figure 9 shows comparison of throughput of the proposed scheduler with other heuristics. The throughput is defined as follows:

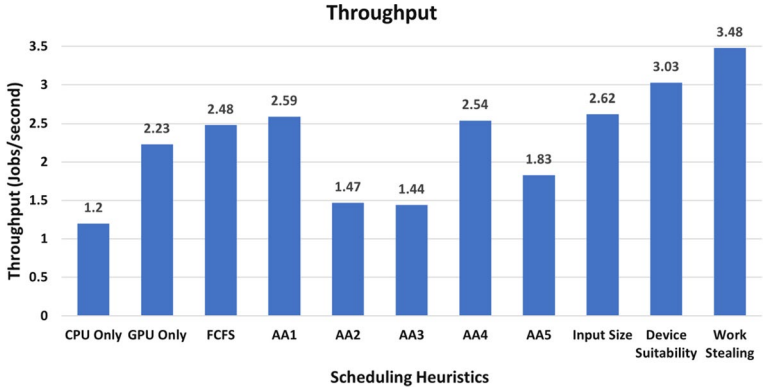


Fig. 9 Throughput of scheduling heuristics

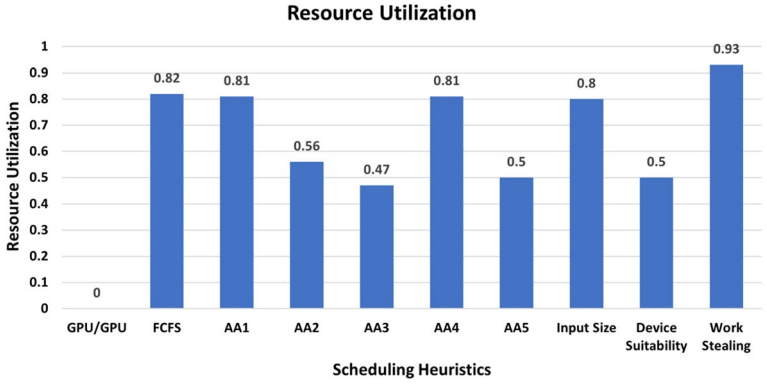


Fig. 10 Resource utilization of scheduling heuristics

$$\text{Throughput} = \text{Total execution time of programs} / \text{total number of programs} \tag{5}$$

Since the total execution time of work-stealing-based scheduler is the minimum, whereas the total number of programs are the same for all alternatives, it results in maximum throughput for our proposed solution.

Figure 10 shows the utilization of devices by different schedulers. It is the measure of time for which a device remains idle. For instance, in case of work stealing, the CPU becomes idle at 32.2 s and remains idle until 34.5 s when GPU completes the assigned tasks. In this case, the idle time of CPU is 2.3 s which is 6.66% of total time of 34.5 s. In this case, we have 100% GPU utilization, whereas CPU utilization is 93.34% as shown in Fig. 10 (0.93). The graph shows that none of the schedulers have more utilization of devices than the proposed scheduler. When all jobs are executed on CPU device only, it means 0% utilization of GPU device since it

remains idle all the time. Similarly, for GPU device only, CPU remains idle all the time. When the jobs are executed using first come first serve scheduler, 82% device utilization is achieved. Similarly, the device utilization is below 93% for all other scheduler which depicts best resource utilization of the proposed solution.

The experimental results in Figs 8, 9, and 10 shows that the proposed scheduling scheme outperforms all other schemes in terms of overall execution time of job, utilization of processing devices and throughput. We believe that better performance of our proposed approach is because of two factors. First, we use machine learning-based predictor to find a suitable device for a task (with less execution time). Second, we use load balancing using work-stealing algorithm that exploit efficient utilization of both (CPU and GPU) devices. The rest of the scheduling heuristics are not considering or combining these two factors while making a scheduling decision. For instance, device suitability is not considered except for device suitability scheduling algorithm. However, device suitability is not incorporating load balancing which make it less efficient than proposed solution.

6 Conclusion and future work

In this research, a novel framework is proposed that is based on machine learning-based job execution time predictor and load balancer using work-stealing algorithm. The programmer maps jobs to processing devices (i.e., CPU and GPU) in a heterogeneous system. However, it is not always straightforward to find a suitable device for a particular task. Therefore, we have proposed a machine learning-based predictor that can predict suitable device with a marginal error rate. We have compared three different machine learning algorithms, and our results demonstrate that random forest performed better with minimum error as compared to other alternatives of multiple linear regression and gradient boosting.

Predicting the device suitability alone is not enough to efficiently utilize the system resources. The mapping of a job to a suitable device may cause offered load imbalance between devices. For instance, one device might be overloaded since it is suitable device for many jobs, and thus, the other device is under loaded and becomes underutilized. The decision of the job's assignment should be balanced to achieve the maximum utilization of devices and reduction in execution time of all jobs. We have demonstrated that work-stealing-based task scheduler balances the workload on both devices and thus achieves much better performance (in terms of total execution time, throughput and device utilization) as compared to other scheduling heuristics.

In our proposed solution, scheduler maps only static jobs to processing devices and considers only the integrated graphics processing units. In future, this framework can be extended to dynamic allocation of jobs to processing devices along the usage of discrete graphic processing units.

Authors' contributions AH is the lead author in this research. He conducts this research work under the supervision of Dr. YNK and Dr. MSR. He performed all experiments and wrote up the first draft. Dr. Y as a supervisor helped AH in formulating research problem statement. In addition, in regular meetings, he helped AH to find out the answers of the following questions. How performance of difference applications should be compared? What are the important performance metrics? Which are the interesting experiments to be performed. He also reviewed the second draft of the paper. The outcome of this review work was the third draft. Dr. Siraj also supervised Mr. A. He helped Asad mainly in paper write up. How this research article should be organized? How to present data in a structured form? What figures and tables are interesting to include in this paper? Where to place different figures and tables? The first draft is written by Asad, which is then extensively reviewed (and rewritten at some places where needed) by Dr. Siraj. The outcome of this work was the second draft. Mr. Muhammad Nadeem Nadir contributed by plotting all figures in this research article. He also proof-read the third draft. The final draft is an outcome of this proof reading.

Funding Not applicable. No funding.

Availability of data and materials Available on request.

Declarations

Conflict of interests I declare that the authors have no competing interests as defined by Springer, or other interests that might be perceived to influence the results and/or discussion reported in this paper.

Ethical Approval Not applicable.

Reference

1. Tsog N, Becker M, Bruhn F, Behnam M, Sjodin M (2019) Static allocation of parallel tasks to improve schedulability in CPU-GPU heterogeneous real-time systems, in: IECON Proceedings (Industrial Electronics Conference), pp. 4516–4522, https://doi.org/10.1109/IECON.2019.8926767.
2. Wen Y, Wang Z, O'Boyle MFP (2014) Smart multi-task scheduling for Open CL programs on CPU/GPU heterogeneous platforms. https://doi.org/10.1109/HiPC.2014.7116910.
3. Ahmed U, Lin JCW, Srivastava G, Aleem M (2021) A load balance multi-scheduling model for OpenCL kernel tasks in an integrated cluster. Soft Comput 25(1):407–420. https://doi.org/10.1007/s00500-020-05152-8
4. Khalid YN, Aleem M, Prodan R, Iqbal MA, Islam MA (2018) E-OSched: a load balancing scheduler for heterogeneous multicores. J Supercomput 74(10):5399–5431. https://doi.org/10.1007/s11227-018-2435-1
5. Ahmed U, Aleem M, Noman Khalid Y, Arshad Islam M, Azhar Iqbal M, RALB-HC: a resource-aware load balancer for heterogeneous cluster, in: Concurrency And Computation: Practice and Experience,https://doi.org/10.1002/cpe.5606.
6. Wenjie T, Yiping Y, Feng Z, Tianlin L, and Xiao S (2017) A work-stealing based dynamic load balancing algorithm for conservative parallel discrete event simulation. https://doi.org/10.1109/WSC.2017.8247833
7. Belviranli ME, Bhuyan LN, Gupt R (2013) A dynamic self-scheduling scheme for heterogeneous multiprocessor architectures. Trans Archit Code Opt. https://doi.org/10.1145/2400682.2400716
8. Choi HJ, Son DO, Kang SG, Kim JM, Lee HH, Kim CH (2013) An efficient scheduling scheme using estimated execution time for heterogeneous computing systems. J Supercomput 65(2):886–902. https://doi.org/10.1007/s11227-013-0870-6
9. Kaleem R, Barik R, Shpeisman T, Lewis BT, Hu C, Pingali K (2014) Adaptive heterogeneous scheduling for integrated GPUs. Parallel Arch Compilat Tech - Conf Proceed, PACT 14:151–162. https://doi.org/10.1145/2628071.2628088
10. Ghose A, Dokara L, Dey S, Mitra P (2017) A framework for OpenCL task scheduling on heterogeneous multicores. Parallel Process Lett. https://doi.org/10.1142/S0129626417500086
11. Moren K, Göhringer D (2018) Automatic mapping for OpenCL-Programs on CPU/GPU heterogeneous platforms. Lect Notes Comput Sci (including subseries Lect Notes in Artif Intell Lecture Notes in Bioinform). https://doi.org/10.1007/978-3-319-93701-4_23

12. Taylor B, Marco VS, Wang Z (2017) Adaptive optimization for OpenCL programs on embedded heterogeneous systems. ACM SIGPLAN Notices 52(5):11–20. https://doi.org/10.1145/3140582.3081040
13. Khalid YN, Aleem M, Ahmed U, Islam MA, Iqbal MA (2019) Troodon: a machine-learning based load-balancing application scheduler for CPU–GPU system. J of Parallel Distributed Comput 132:79–94. https://doi.org/10.1016/j.jpdc.2019.05.015
14. Grewe D, O'Boyle MFP (2011) A static task partitioning approach for heterogeneous systems using OpenCL. Lecture Notes Comput Sci. https://doi.org/10.1007/978-3-642-19861-8_16
15. Boyer M, Skadron K, Che S, Jayasena N (2013) Load balancing in a changing world: dealing with heterogeneity and performance variability. https://doi.org/10.1145/2482767.2482794.
16. Huchant P, Counilh MC, Barthou D (2016) Automatic OpenCL task adaptation for heterogeneous architectures. Lect Notes Comput Sci. https://doi.org/10.1007/978-3-319-43659-3_50
17. Wang Z, Zheng L, Chen Q, and Guo M (2013) CAP: Co-scheduling based on asymptotic profiling in CPU+GPU hybrid systems, in: Proceedings of the 2013 International Workshop on Programming Models And Applications for Multicores and Manycores, PMAM 2013, pp. 107–114. https://doi.org/10.1145/2442992.2443004.
18. Nozal R, Bosque JL, Beivide R (2020) EngineCL: Usability and performance in heterogeneous computing. Futur Gener Comput Syst 107:522–537. https://doi.org/10.1016/j.future.2020.02.016
19. Lee J, Samadi M, and Mahlke S (2015) Orchestrating multiple data-parallel kernels on multiple devices. https://doi.org/10.1109/PACT.2015.14.
20. Memeti S and Pllana S (2018) HSTREAM: A directive-based language extension for heterogeneous stream computing, Proceedings - 21st IEEE International Conference on Computational Science and Engineering, CSE 2018, pp. 138–145. https://doi.org/10.1109/CSE.2018.00026.
21. Kumar V, Sbîrlea A, Jayaraj A, Budimlić Z, Majeti D, and Sarkar V (2015) heterogeneous work-stealing across CPU and DSP cores, 2015 IEEE High Performance Extreme Computing Conference, HPEC 2015, https://doi.org/10.1109/HPEC.2015.7322452.
22. Agostini M, O'Brien F, Abdelrahman T (2020) Balancing Graph processing workloads using work stealing on heterogeneous CPU-FPGA Systems. ACM Int Conf Proceed Series. https://doi.org/10.1145/3404397.3404433
23. Wen Y, O'Boyle MFP (2017) Merge or separate? multi-job scheduling for OpenCL Kernels On Cpu/Gpu Platforms, in: proceedings of the General Purpose GPUs, GPGPU-10 2017, pp. 22–31, https://doi.org/10.1145/3038228.3038235.
24. Liu X, Ounifi HA, Gherbi A, Li W, Cheriet M (2020) A hybrid GPU-FPGA based design methodology for enhancing machine learning applications performance. J Ambient Intell Humaniz Comput 11(6):2309–2323. https://doi.org/10.1007/s12652-019-01357-4
25. Memeti S, Pllana S (2021) Optimization of heterogeneous systems with AI planning heuristics and machine learning: a performance and energy aware approach. Computing 103(12):2943–2966. https://doi.org/10.1007/s00607-021-01017-6
26. Rahmani TA, Daham F, Belalem G, Mahmoudi SA, HBalancer: A machine learning based load balancer in real time CPU-GPU heterogeneous systems, in: 2022 International Conference on Innovation and Intelligence for Informatics, Computing, and Technologies, 3ICT 2022, pp. 674–679, 2022, https://doi.org/10.1109/3ICT56508.2022.9990623.
27. Alsubaihi S and Gaudiot JL (2017) A runtime workload distribution with resource allocation for CPU-GPU heterogeneous systems,in: Proceedings - 2017 IEEE 31st International Parallel and Distributed Processing Symposium Workshops, IPDPSW 2017, pp. 994–1003, https://doi.org/10.1109/IPDPSW.2017.19.
28. Lee J, Samadi M, Park Y, Mahlke S (2015) SKMD: Single kernel on multiple devices for transparent CPU-GPU collaboration. ACM Trans Comput Syst. https://doi.org/10.1145/2798725
29. Becchi M, Byna S, Cadambi S, and Chakradhar S (2010) Data-aware scheduling of legacy kernels on heterogeneous platforms with distributed memory, in: Annual ACM Symposium on Parallelism In Algorithms and Architectures, pp. 82–91. https://doi.org/10.1145/1810479.1810498.
30. Munshi A (2016) The OpenCL specification, 2009 IEEE Hot Chips 21 Symposium, HCS 2009, pp. 11–314, https://doi.org/10.1109/HOTCHIPS.2009.7478342.
31. Wang Z and O'Boyle M (2018) "Machine learning in compiler optimization, https://doi.org/10.1109/JPROC.2018.2817118.
32. Wang Y, Ji W, Shi F, Zuo Q (2013) A work-stealing scheduling framework supporting fault tolerance. https://doi.org/10.7873/date.2013.150
33. "Scheduler — CAF 0.17.5 documentation."

34. Kreiliger F, Matejka J, Sojka M, Hanzálek Z (2019) Experiments for predictable execution of GPU Kernels. Ospert 2019:23
35. Daga M, Aji AM, Feng WC (2011) On the efficacy of a fused CPU+GPU processor (or APU) for parallel computing,in: Proceedings - 2011 Symposium On Application Accelerators in High-Performance Computing, SAAHPC 2011, pp. 141–149,https://doi.org/10.1109/SAAHPC.2011.29
36. Manathunga M, Aktulga HM, Götz AW, Merz KM (2023) Quantum mechanics/molecular mechanics simulations on NVIDIA and AMD graphics processing units. J Chem Inf Model 63(3):711–717. https://doi.org/10.1021/ACS.JCIM.2C01505
37. Weinhardt M (2022) An Analysis of Mapping Polybench Kernels to HPC CGRAs, Proceedings - 2022 IEEE 36th International Parallel and Distributed Processing Symposium Workshops, IPDPSW 2022, pp. 647–654, https://doi.org/10.1109/IPDPSW55747.2022.00114
38. Öz I, Karadaş ÖF (2022) Regional soft error vulnerability and error propagation analysis for GPGPU applications. J Supercomput 78(3):4095–4130. https://doi.org/10.1007/s11227-021-04026-6

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Asad Hayat[4] · Yasir Noman Khalid[3] · Muhammad Siraj Rathore[1] · Muhammad Nadeem Nadir[2]

✉ Muhammad Siraj Rathore
muhammad.siraj@cust.edu.pk

Asad Hayat
asad.cs@leads.edu.pk

Yasir Noman Khalid
yasir.noman.khalid@hitecuni.edu.pk

Muhammad Nadeem Nadir
nadeem.nadir@cs.uol.edu.pk

[1] Department of Computer Science, Capital University of Science and Technology, Islamabad 44000, Pakistan

[2] Department of Computer Science, University of Lahore, Lahore, Pakistan

[3] Department of Computer Science, Hitech University, Taxila, Pakistan

[4] Department of Computer Science, Lahore Leads University, Lahore, Pakistan

View publication stats